SYSTEM DESIGN: WEEK 4

Advanced UML Diagrams, Technical Design Documents, and Architecture Decision Records (ADRs)

Advanced UML Diagrams

Unified Modeling Language (UML) is a standardized visual language used to represent, design, and understand software systems before or during development.

Three Important UML Diagrams:

1. **Use-case Diagram:** Shows how external users interact with the system.
   Example: A customer places an order.
2. **Activity Diagram:** Shows the steps and decisions in a workflow.
   Example: Validate cart → Process payment → Confirm order.
3. **State Machine Diagram:** Shows an object's states and how events change them.
   Example: An order changes from Pending → Paid.

Key Difference: Use-case shows who does what, Activity shows how a process works, and State Machine shows how an object changes state.

Aspect	Point	Description
Common Notation	Shared visual language	Helps developers, designers, testers, and stakeholders communicate.
Different Diagrams	Each has a purpose	Use-case, activity, and state machine diagrams answer different questions.
Missing Requirements	Reveals gaps	Helps identify missing steps, errors, actors, and state changes.
Complements Code	Supports documentation	Explains system design but does not replace code or specifications.
Appropriate Detail	Keep it clear	Include enough detail to answer the question without unnecessary complexity.

Practical example: Consider an online shopping application. Before writing code, a team might create:

1. **Use-case Diagram** → Shows user interactions.
2. **Activity Diagram** → Shows process steps.
3. **State Machine Diagram** → Shows changes in object states.

Benefit: These diagrams clarify system behavior before developing the database, APIs, and user interface.

Use Case Diagrams

A **Use-Case Diagram** is a UML behavioral diagram that shows interactions between external actors and system functions. It describes what the system does, not how it works internally.
Example: A student searches, borrows, and returns books using a library system.

Five Key Points

1. **Actors** → Represent users or external systems, e.g., Student, Librarian, Payment Gateway.
2. **Use Cases** → Represent system functions, e.g., Search Book, Make Payment.
3. **System Boundary** → A rectangle defining the system; use cases appear inside it.
4. **Relationships** → Show connections between actors and use cases.
5. **Functional Scope** → Shows system functionality without detailed algorithms or database design.

Important Use-Case Relationships

- **Association** → Connects an actor to a use case.
  Example: Student — Register for Event.
- **<<include>>** → Represents behavior included as part of another use case.
  Example: Checkout includes Process Payment.
  Arrow direction: Including use case → Included use case.
- **<<extend>>** → Represents optional or conditional behavior added to a base use case.
  Example: Apply Coupon extends Checkout.
  Arrow direction: Extending use case → Base use case.
- **Generalization** → Represents an inheritance-like relationship.
  Example: Administrator and Librarian are specialized types of Staff Member.
  Arrow direction: Child → Parent (hollow triangle).

Important: Use include for incorporated behavior and extend for conditional additional behavior.

Practical Example: College Event Registration System

Actors → Student, Notification Service
Use Cases →Browse Events, Register for Event, Cancel Registration, View Registration Status, Send Confirmation
Relationships→ Student interacts with the first four use cases, Register for Event <<include>> Send Confirmation, Notification Service participates in sending confirmations.

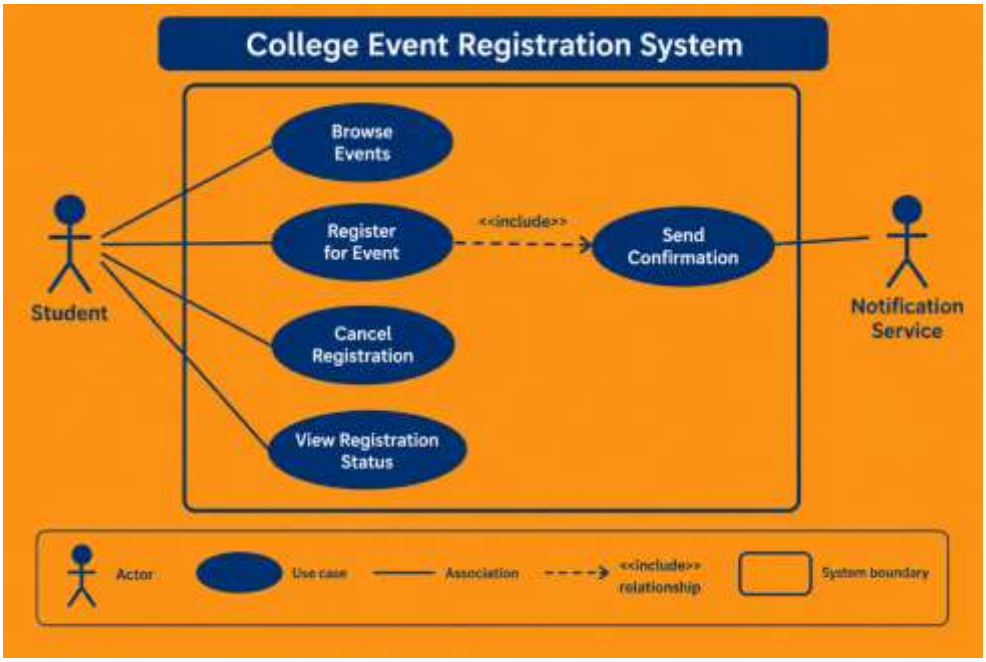
Use-case diagram

How to interpret the diagram

1. The student is an external actor.
2. The rectangle defines the system boundary.
3. The four student-facing use cases represent available functionality.
4. Registration includes sending a confirmation, according to the assumptions of this example. The notification service is external to the application.

The following PlantUML code generates a UML use-case diagram.

```
@startuml
left to right direction
actor Student
actor "Notification Service" as Notification
rectangle "College Event Registration System" {
   usecase "Browse Events" as UC1
   usecase "Register for Event" as UC2
   usecase "Cancel Registration" as UC3
   usecase "View Registration Status" as UC4
   usecase "Send Confirmation" as UC5
}
Student -- UC1
Student -- UC2
Student -- UC3
Student -- UC4
UC2 ..> UC5 : <<include>>
Notification -- UC5
@enduml
```



Example using <<extend>>

Suppose a shopping application allows customers to apply coupons during checkout. Here, payment processing is included in checkout, while coupon application extends checkout.

The coupon behavior is conditional or optional, whereas the modeled payment-processing behavior is part of the checkout workflow.

```
@startuml
left to right direction
actor Customer
rectangle "Online Shopping System" {
   usecase "Checkout" as Checkout
   usecase "Apply Coupon" as Coupon
   usecase "Process Payment" as Payment
}
Customer -- Checkout
Checkout ..> Payment : <<include>>
Coupon ..> Checkout : <<extend>>
@enduml
```

Activity Diagram

An Activity Diagram is a UML behavioral diagram that shows the flow of activities in a process. It explains how a process progresses from one action to another**.**

Five Key Points

1. **Actions** → Represent operations using rounded rectangles.
2. **Control Flows** → Arrows show the direction of activities.
3. **Decisions and Merges** → Diamonds represent branching or merging paths.
4. **Forks and Joins** → Thick bars represent parallel activities and their synchronization.
5. **Swimlanes** → Show the responsibilities of different actors or departments.

Symbol	Meaning	Example
Filled circle	Initial node	Start checkout
Rounded rectangle	Action	Validate cart
Diamond	Decision or merge	Payment successful?
Thick bar	Fork or join	Start or synchronize parallel work
Bullseye-like final symbol	Activity final	End the workflow
Arrow	Control flow	Move to the next action
Swimlane	Responsibility partition	Customer, Order Service, Payment Service

Practical example: Online order checkout

Suppose a customer purchases a product.
The workflow is:
(Start) → [Validate Cart] → Cart Valid? → **No:** [Show Error] → (End) | **Yes:** [Calculate Total] → [Process Payment] →Payment Successful? → **Yes:** [Confirm Order] → (End) | **No:** [Show Payment Failure] → (End)

```
@startuml
 start
 :Validate cart;
  if (Cart valid?) then (Yes)
  :Calculate total;
  :Process payment;
  if (Payment successful?) then (Yes)
     :Confirm order;
     :Send confirmation;
  else (No)
     :Display payment failure;
  endif
  else (No)
  :Display cart error;
  endif
  stop
 @enduml
```

Swimlanes

Swimlanes are useful when several actors or system components participate in the same workflow.

For example:

- Customer
- Order Service
- Payment Service

A swimlane diagram makes responsibility explicit.

```
@startuml
|Customer|
start
:Submit order;
|Order Service|
:Validate order;
|Payment Service|
:Process payment;
if (Payment successful?) then (Yes)
|Order Service|
   :Confirm order;
|Customer|
   :Receive confirmation;
else (No)
|Order Service|
   :Mark payment as failed;
|Customer|
  :Receive failure message;
endif
 stop
@enduml
```

Parallel activities

After order confirmation, two tasks can run concurrently:

- **Send Notification** → Notify the customer.
- **Update Analytics** → Record order data.

Note → Parallel execution is possible if system consistency requirements allow it.

```
@startuml
start
:Confirm order;
 fork
   :Send confirmation notification;
fork again
   :Record analytics event;
end fork
:Finish workflow;
stop
@enduml
```

1. **Fork** → Starts parallel activities.

2. **Join** → Waits for all parallel activities to finish.
3. **Asynchronous Processing** → Allows tasks like notifications and analytics to run independently without delaying the main workflow.

Coding and Modeling Problem

Problem: Student Assignment Submission

Create an activity diagram showing:

- Student selects an assignment and the system checks the deadline.
- Student uploads a file, and the system validates its type and size.
- Invalid deadlines or files result in rejection; valid submissions receive confirmation.

Challenge → The file must be saved and recorded in the database before confirmation.
Decide whether these operations should run sequentially or in parallel.

State Machine Diagrams

A **State Machine Diagram** is a UML behavioral diagram that shows an object's states and transitions during its lifecycle.
Example → An order moves from Pending to Paid when payment succeeds.

Five Key Points

1. **States** → Represent an object's current condition.
2. **Transitions** → Show movement between states.
3. **Events** → Trigger transitions, e.g., paymentSucceeded.
4. **Guard Conditions** → Conditions that must be true for a transition.
5. **Entry/Exit Actions** → Actions performed when entering or leaving a state.

State vs. Event

1. **State** → Pending (current condition).
2. **Event** → paymentSucceeded (something that happens).
3. **Transition** → Pending → Paid.
4. **Guard Condition** → [paymentAmount == orderTotal] (payment must match the order total).

Example → Pending -- paymentSucceeded [amount valid] --> Paid

Practical Example: E-commerce Order Lifecycle

Order Lifecycle → Pending → Paid → Shipped → Delivered

Cancellation Rules →

- **Pending** → Can be cancelled.
- **Paid** → Can be cancelled before the allowed fulfilment stage.

- **Shipped** → Cannot be cancelled normally.
- **Delivered** → Terminal state in this model.

State machine diagram

```
@startuml
[*] --> Pending
 Pending --> Paid : paymentSucceeded
 Pending --> Cancelled : cancelRequested
 Paid --> Shipped : shipmentDispatched
 Paid --> Cancelled : cancellationApproved
 Shipped --> Delivered : deliveryConfirmed
 Cancelled --> [*]
 Delivered --> [*]
 @enduml
```

Guard conditions

Suppose a paid order can be cancelled only if the order has not been dispatched.
The transition can be represented as:

```
@startuml
[*] --> Pending
Pending --> Paid : paymentSucceeded
Paid --> Cancelled : cancelRequested [not
dispatched]
Paid --> Shipped : shipmentDispatched
Shipped --> Delivered : deliveryConfirmed
Delivered --> [*]
Cancelled --> [*]
```

The guard [not dispatched] expresses a condition that must be satisfied for the transition.
A guard is not an action. It determines whether the transition is permitted.

Entry and exit actions

Some UML state machines allow actions to be associated with entering or leaving a state.
For example, entering Paid might trigger an internal action to record a payment confirmation.

```
@startuml
[*] --> Pending
Pending --> Paid : paymentSucceeded
state Paid {
   entry / Record payment confirmation
}Paid --> Shipped : shipmentDispatched
@enduml
```

State machine versus activity diagram

Although both diagrams describe behavior, they serve different purposes.

Activity diagram	State machine diagram
Focuses on a workflow or process	Focuses on an object's lifecycle
Emphasizes actions and control flow	Emphasizes states and state transitions
Useful for checkout steps	Useful for order status changes
Can represent parallel activities	Can represent event-driven state changes
Answers, "What happens next in this process?"	Answers, "What state is the object in, and how can it change?"

Coding and Modeling Problem: ATM Withdrawal

Requirements → Model ATM states from Idle to Completed or Failed. Allow withdrawals only after successful authentication, with a valid amount and sufficient funds.
Challenge → If cash is not dispensed after bank approval, the system may require a transaction reversal or reconciliation.

Writing Technical Design Documents

A **Technical Design Document (TDD)** explains how a software system or feature will be implemented, providing a shared plan for developers and testers.
It covers the problem, requirements, architecture, technologies, alternatives, testing, deployment, and risks.

Five Key Points

1. **Define the Problem** → Explain what needs to be solved and why.
2. **Separate Requirements and Design** → Requirements describe *what* the system does; design explains how.
3. **Explain Architecture** → Describe components, responsibilities, and data flow.
4. **Document Trade-offs** → Explain the chosen solution and its disadvantages.
5. **Consider Operations** → Include security, testing, deployment, monitoring, and recovery.

Practical Example: College Assignment Portal

A TDD for an assignment portal describes student uploads, teacher reviews, deadlines, frontend, backend, database, file storage, validation, testing, deployment, and upload failure handling.

Benefit → Provides a clear implementation plan before development begins.

Structure of a Technical Design Document

Section	What to Include	Example
Title and metadata	Feature name, author, status, reviewers, date	Assignment Submission Portal
Problem statement	Current problem and why it matters	Assignments are collected manually
Goals	Outcomes the design must achieve	Support online submission
Non-goals	Features intentionally excluded	AI-based assignment grading
Requirements	Functional and non-functional requirements	Upload files, validate deadlines
High-level architecture	Components and their relationships	Frontend, backend, database, file storage
Detailed design	APIs, data models, algorithms, validation rules	POST /assignments/{id}/submissions
Security	Authentication, authorization, validation, data protection	Only enrolled students can submit
Alternatives	Other solutions and their trade-offs	Local file storage versus object storage
Testing	Unit, integration, and end-to-end tests	Reject uploads after the deadline
Monitoring	Metrics, logs, alerts, error tracking	Upload failures and request latency
Rollout and migration	Deployment stages, compatibility, rollback	Deploy to a pilot group
Risks and open questions	Unresolved assumptions and potential failures	Maximum allowed file size

Problem Statement

A **problem statement** explains the issue, who is affected, its consequences, and the desired outcome.
Weak Example → "We need an assignment website."
Strong Example → Students submit assignments through multiple channels, making it difficult to track deadlines and missing work. The system will centralize submissions and track their status.

Goals and Non-Goals

Goals → Features the project aims to provide.

- Allow online assignment submissions.
- Validate deadlines.
- Let teachers view submissions.
- Track submission status.

Non-Goals → Features excluded from the project.

- AI-based grading.
- Live online examinations.
- External LMS integration in the first version.

Importance → Non-goals prevent scope confusion and make planning realistic.

Functional and Non-Functional Requirements

Functional Requirements → Describe what the system does.

- Students can upload assignments.
- Teachers can view submissions.
- The system rejects oversized files.

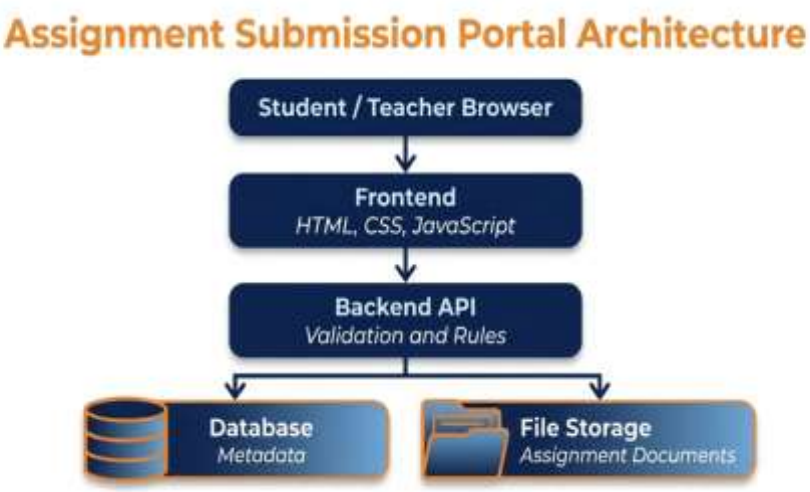
Non-Functional Requirements → Describe system quality and constraints.

- Protect student records.
- Meet response-time targets.
- Log failed uploads.
- Preserve records after application restarts.

Key Point → Requirements should be measurable whenever possible. For example, instead of saying "the system should be fast," specify that the API must respond within 500 milliseconds at the 95th percentile under an agreed test load.

High-level architecture

High-level architecture describes the major components and their interactions.



Detailed Design

Detailed Design → Explains how developers should implement the system. It includes APIs, database design, classes, validation, error handling, security, and data consistency.

Example: Assignment Submission API

- **Endpoint** → POST /assignments/{assignmentId}/submissions
- **Purpose** → Submit an assignment.
- **Validation** → Check deadline, file type, size, and student authorization.
- **Success** → Save the submission and return its ID and status.
- **Failure** → Return an error without marking the submission successful.

Security Considerations

Security → Protects the system and its data.

- **Authentication** → Verify user identity.

- **Authorization** → Restrict access based on permissions.
- **Input Validation** → Validate files and inputs.
- **File Security** → Store uploaded files safely.
- **Data Protection** → Protect sensitive information.
- **Auditability** → Record important actions.

Monitoring and Observability

Monitoring → Checks system performance and detects problems after deployment.

- **Availability** → Successful health checks.
- **Latency** → API response time.
- **Errors** → Failed uploads.
- **Capacity** → Storage usage.
- **Reliability** → Database failures.
- **Business Operations** → Accepted and rejected submissions.

Logs → Explain events.
Metrics → Show performance trends.
Traces → Track requests across services.

Small projects may need basic logs, while larger systems may require dashboards and alerts.

Coding and Documentation Problem

Problem → Design a Library Management System that supports searching, borrowing, and returning books, tracking due dates, and calculating fines.
Include →

1. Problem statement and goals.
2. Requirements and architecture.
3. Database and API design.
4. Security, testing, and monitoring.
5. Alternative solutions and rollout/rollback plans.

Challenge → Explain how the design changes to support multiple library branches.
What It Tests → Ability to convert requirements into a technical implementation plan.

Evaluating Technical Alternatives

Evaluating alternatives means comparing technical solutions based on requirements and constraints. The design document should explain why a solution was selected and its trade-offs.

Five Key Points

1. **Define criteria:** Compare cost, performance, reliability, and maintainability.
2. **Consider alternatives:** Evaluate solutions that meet requirements.
3. **Document trade-offs:** Improving one quality may affect another.
4. **Use evidence:** Consider benchmarks, prototypes, and documentation.
5. **Avoid false precision:** Use meaningful scores and assumptions.

Example: Choosing a Database

A college event application stores student details, events, and registrations.

- **Relational database:** Suitable for structured data and relationships.
- **Document database:** Suitable for flexible document structures.

The choice depends on actual requirements, not popularity.

Comparison table

Criterion	Relational database	Document database
Data model	Tables and relationships	Documents and collections
Relationships	Supports joins and relational constraints	Often uses embedded documents or references
Schema	Explicit schema, with supported evolution mechanisms	Often flexible document structures
Transactions	Supports transactions, depending on the database	Transaction support varies by product and deployment
Best fit	Structured, relational data and integrity requirements	Document-centric data and suitable flexible-schema workloads
Main consideration	Schema changes and query design require planning	Data duplication and relationship management may require additional care

Decision Matrix: Assign weights to criteria and score each option from 1–5.

Decision Matrix

Criterion	Weight	Option A	Option B
Data integrity	5	5	3
Ease of implementation	3	4	4
Query requirements	4	5	3
Operational complexity	2	4	3
Weighted total		**61**	**39**

Formula: Weighted score = Weight × Score, summed across all criteria.

Option A scores higher under these hypothetical assumptions. Results depend on the chosen weights and requirements.

Coding and Design Problem

Problem: Choose Storage for a Daily Activity Tracker

The application stores activities, due dates, completion status, and categories.

Options:

1. Local file storage.

2. Relational database.
3. Cloud-hosted database.

Requirements:

- Compare ease of development, data persistence, multi-device access, offline support, and deployment complexity.
- Recommend an option for a beginner-level college project.
- Explain two disadvantages of your choice.

Challenge: Why might a local prototype and a multi-user production application require different storage solutions?

Concept tested: Requirements-based decisions, trade-off analysis, and avoiding technology choices based only on popularity.

Rollout and Migration Plans

A rollout plan explains how a new system is introduced, while a migration plan explains how existing data or functionality is moved to a new system.

Five Key Points

1. **Plan deployment:** Use testing, pilot release, and full deployment.
2. **Maintain compatibility:** Ensure old and new versions work together.
3. **Protect data:** Preserve records and data integrity.
4. **Plan recovery:** Define rollback procedures.
5. **Verify results:** Monitor errors, performance, and data consistency.

Example: Migrating an Assignment Portal

Moving data from local files to a relational database involves: Back up existing data, create the database schema, import and validate records, test the application, release gradually while monitoring results, and maintain a recovery plan.

Backward-Compatible API Migration

Old response:

```
{
  "studentName": "Aarav"
}
```

New response:

```
{
  "studentName": "Aarav",
  "department": "Information Technology"
    }
```

a field is generally backward-compatible, but client behavior must be checked.

Steps: Check existing clients, add the field, deploy and monitor, update clients, and remove the old field only when no longer needed.

Rollback Strategies

Strategy	How It Works	Main Consideration
Redeploy previous version	Restore the previous release	Database compatibility
Feature flag	Disable the new feature	Must support safe disabling
Database restoration	Restore an earlier state	May lose newer data
Forward fix	Deploy a correction	Reversing changes may be unsafe
Blue-green deployment	Switch between two environments	Requires compatibility planning
Canary rollout	Release to a small group first	Requires monitoring

Problem: Plan the migration of a student database into separate student and department tables, covering data backup, schema changes, record validation, compatibility, rollback, recovery, and preventing data loss during new registrations.

Concept tested: Data migration, compatibility, deployment, and recovery.

Architecture Decision Records (ADRs)

An **Architecture Decision Record (ADR)** documents an important architectural decision, its context, alternatives, and consequences. It helps future developers understand why a decision was made.

Five Key Points

1. **One decision per ADR:** Focus on a single architectural decision.
2. **Explain the context:** Record requirements and constraints.
3. **State the decision:** Clearly identify the chosen approach.
4. **Document trade-offs:** Explain benefits and disadvantages.
5. **Preserve reasoning:** Record why the decision was appropriate at that time.

Practical Example

A college event application needs a database. The team chooses a relational database because student registrations are linked to events and require data consistency. The ADR records the decision and its trade-offs.

ADR vs Technical Design Document

Technical Design Document	Architecture Decision Record
Describes system design and implementation	Records a specific architectural decision
Covers multiple components and workflows	Focuses on one decision
Includes APIs, data, security, and testing testing	Includes context, alternatives, and consequences
Supports implementation planning	Explains architectural reasoning
May evolve with implementation	Preserves historical reasoning and status

ADR Lifecycle: Proposed, accepted, rejected, deprecated, or superseded; superseded ADRs remain available for historical reference.

Problem: Choose a backend framework for a college event-management application by defining the problem and constraints, comparing alternatives, documenting the decision's benefits and disadvantages, and identifying when to reconsider it.

Challenge: Explain why popularity alone is insufficient for choosing a technology.

Concept tested: Decision documentation, trade-offs, and context-based technical choices.

Writing an Effective ADR

Standard ADR Structure

An ADR typically includes:

1. Number, Title, Status, and Date
2. Context: Problem and constraints.
3. Decision: Selected approach.
4. Alternatives: Other options and reasons for rejection.
5. Consequences: Benefits, disadvantages, and risks.
6. Reconsideration Conditions: When to review the decision.

Example ADR: Choosing a Relational Database

ADR-001: Select a Relational Database

Status: Accepted
Date: 24 October 2026

1. Context: A college event application must manage students, events, and registrations while maintaining data consistency.
2. Decision: Use a relational database with primary keys, foreign keys, and transactions.
3. Alternatives:

- **Relational database:** Supports relationships, constraints, and structured queries.
- **Document database:** Offers flexible structures but may require additional relationship management.

4. Consequences:

- **Benefits:** Consistent data, clear relationships, and reliable queries.
- **Disadvantages:** Requires schema maintenance and migration planning.

5. Reconsideration: Review the decision if requirements, performance, or operational needs change.

Why is this a good ADR? It explains the decision, compares alternatives, documents trade-offs, and identifies when reconsideration is necessary.

Context, Decision, and Consequences

The three main parts of an ADR explain why a technical decision was made and its effects.

Context: Explains the problem and constraints, such as requirements, budget, security, performance, and team expertise.

Example: A college application needs email notifications with a small budget and low message volume.

Decision: Clearly states the selected approach.

Example: Use a background job queue to send confirmation emails asynchronously without delaying registration requests.

Consequences: Describes the benefits and disadvantages of the decision.

- **Benefits:** Faster registration, automatic retries, and independent scaling.
- **Disadvantages:** Additional infrastructure, possible delays, and duplicate-message handling.

Importance of Documenting Consequences

Documenting consequences helps teams understand trade-offs and decide whether a solution is suitable.For example, asynchronous notifications may be delayed, but registration records must remain accurate.

Coding and Documentation Problem

Problem: Compare synchronous email delivery with asynchronous delivery using a background queue.

Requirements: Explain request latency, provider failures, infrastructure, failed-job handling, and when to reconsider the decision.

Challenge: How will you ensure registrations are saved even when email delivery fails?

Concept tested: Reliability, consistency, asynchronous processing, and ADR writing.

Integrated Case Study: E-Commerce Order System

This case study combines **Advanced UML Diagrams**, **Technical Design Documentation**, and **Architecture Decision Records (ADRs).**

Problem Statement

An e-commerce application must allow customers to place orders, make payments, track orders, and cancel them when permitted. It must handle payment failures and prevent invalid order-state transitions.

Goals

- Accept valid orders and verify payments.
- Track order status and support cancellations.
- Notify customers about important events.
- Maintain consistent order records.

Non-Goals

The initial version excludes international currency conversion, recurring subscriptions, loyalty points, and AI-based recommendations. These may be added later.

Use-case diagram

The main actors are:

- Customer.
- Payment Gateway.
- Delivery Service.

The system provides the following use cases:

- Place Order.
- Make Payment.
- Track Order.
- Cancel Order.
- Dispatch Shipment.

```
@startuml
left to right direction
actor "Payment Gateway" as Payment
actor "Delivery Service" as Delivery
rectangle "E-Commerce Order System" {
    usecase "Place Order" as Place
    usecase "Make Payment" as Pay
    usecase "Track Order" as Track
    usecase "Cancel Order" as Cancel
    usecase "Dispatch Shipment" as Ship
}
Customer -- Place
Customer -- Track
Customer -- Cancel
Place ..> Pay : <<include>>
Payment -- Pay
Delivery -- Ship

@enduml
```

Activity diagram

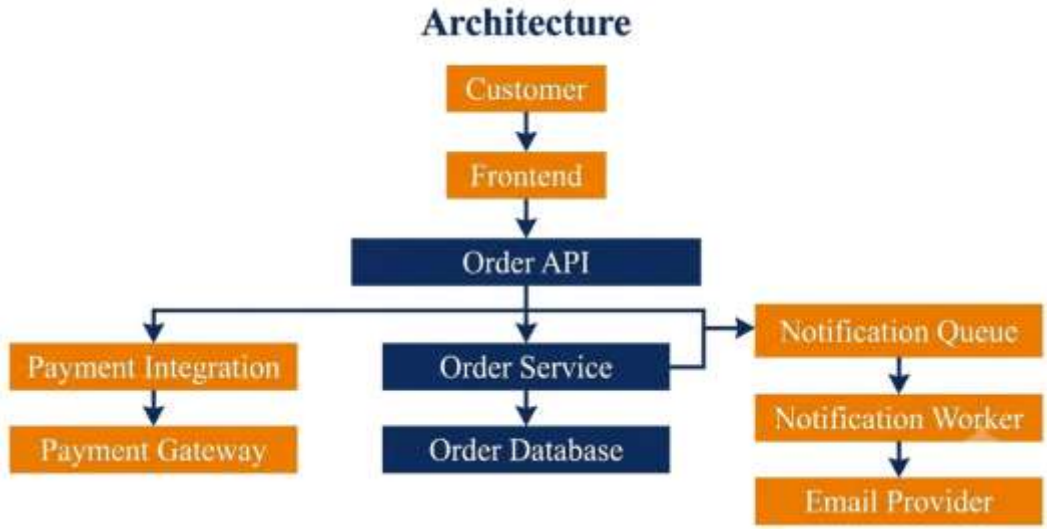
The activity diagram describes the checkout process.

```
@startuml
start
:Receive checkout request;
:Validate order details;
if (Order valid?) then (Yes)
    :Calculate total;
    :Request payment;
if (Payment successful?) then (Yes)
        :Persist confirmed order;
        :Schedule fulfilment;
        :Send confirmation;
else (No)
        :Record payment failure;
        :Display payment failure;
endif
else (No)
:Display validation errors;
endif
stop
@enduml
```

State machine diagram

```
@startuml
[*] --> Pending
Pending --> Paid : paymentSucceeded
Pending --> Cancelled : cancelRequested
Paid --> Shipped : shipmentDispatched
Paid --> Cancelled : cancellationApproved
Shipped --> Delivered : deliveryConfirmed
Delivered --> [*]
Cancelled --> [*]
@enduml
```

Technical design summary



Example ADR: Use Asynchronous Notifications

ADR-002: Send Order Notifications Asynchronously

Context: Email delays or failures should not affect successful order processing.

Decision: Save the order and reliably create a notification job for asynchronous processing.

Alternatives: Synchronous email sending or asynchronous queue-based delivery.

Consequences:

- **Benefits:** Independent retries, faster order processing, and separate scaling.
- **Drawbacks:** Additional infrastructure, possible delays, and duplicate or failed-job handling.

Reconsideration: Review the decision if notification volume, reliability, or infrastructure requirements change.

What This Case Study Teaches

- **Use-case diagram:** Shows actors and system functions.
- **Activity diagram:** Explains the checkout workflow.
- **State machine diagram:** Defines valid order-state transitions.
- **Technical design document:** Explains implementation details.
- **ADR:** Records architectural decisions and their reasoning.

Together, these artifacts provide a complete view of the system's design.

Practice Questions for Week 4

Use these questions to test whether you understand the concepts well enough to apply them in a new system.

A. Conceptual questions

1. What is UML, and why is it useful during software design?
2. What is the difference between a use-case diagram and an activity diagram?
3. What is the difference between an activity diagram and a state machine diagram?
4. What is the purpose of a system boundary in a use-case diagram?
5. Explain the difference between <<include>> and <<extend>>.
6. What is the difference between a decision node and a fork node?
7. What is a swimlane, and why is it useful?
8. What is the difference between a state and an event?
9. What is a guard condition in a state machine?
10. What is the difference between a Technical Design Document and an ADR?
11. Why should a technical design document contain non-goals?
12. What is the difference between functional and non-functional requirements?
13. Why should technical alternatives be evaluated before implementation?
14. What is the difference between rollout and migration?
15. Why is backward compatibility important during deployment?
16. What information should an ADR contain?
17. Why should an ADR record negative consequences as well as benefits?
18. What is the difference between a proposed and a superseded ADR?

Why is a rollback plan not always sufficient to recover from a database migration?

Diagram Selection Matrix

Situation	Situation	Correct diagram
Identifying what students and teachers can do in a college portal		Use-case diagram
Showing the steps involved in submitting an assignment		Activity diagram
Showing how an order moves from Pending to Paid to Shipped		State machine diagram
Identifying the external actors interacting with a system		Use-case diagram
Representing parallel processing of two independent tasks		Activity diagram
Defining which events permit an order-state transition		State machine diagram
Showing which department is responsible for each workflow step		Activity diagram with swimlanes

Identify the documentation artifact

Situation	Appropriate artifact
Describing a feature's architecture, API, database, and security	Technical Design Document
Recording why a relational database was selected	ADR
Explaining which features are excluded from the current project	Technical Design Document
Recording the disadvantages of asynchronous notifications	ADR
Defining deployment stages and rollback steps	Technical Design Document or deployment plan
Recording that a previously accepted architectural decision has been replaced	Superseded ADR, linked to its replacement
Explaining how a new API will remain compatible with old clients	Technical Design Document
Comparing alternative caching technologies and documenting the selected approach	Technical Design Document, ADR, or both, depending on scope

Advanced design questions

1. Why might an activity diagram be insufficient to describe an order's complete lifecycle?
2. How would you model a payment that is initially processing but later succeeds or fails?
3. Can an optional feature be represented using <<include>>? Explain the semantic distinction.
4. Why should a state machine explicitly represent failure states when failure affects later operations?
5. How can an activity diagram help identify missing error-handling logic?
6. What should a technical design document say about a requirement that is still uncertain?
7. How would you compare two caching solutions without relying solely on performance claims?
8. When should an ADR be reconsidered?

9. What problems arise if a team updates its architecture but does not update its documentation?
10. Why might a feature flag be safer than rolling back an entire application deployment?
11. How would you migrate a database while keeping an older application version operational?
12. What is the difference between documenting an architectural decision and documenting its implementation?
13. How would you model an order that can be returned after delivery?
14. Why might a database migration require a forward fix rather than a simple rollback?
15. How can UML diagrams and ADRs complement one another?

Answer Key: Important Concepts

1. Why might an activity diagram be insufficient to describe an order's complete lifecycle?

An activity diagram focuses on the flow of actions through a process. It can describe checkout, but it does not necessarily define all the valid states an order may occupy or the events that permit transitions between them.

A state machine is more suitable for expressing the order's lifecycle and its transition rules.

2. Can an optional feature be represented using <<include>>?

An include relationship represents behavior incorporated into another use case. An optional behavior that extends a base use case under specified conditions is generally better represented with <<extend>>.

However, whether a use case is optional in the business process and how its behavior is modeled must be considered together. The relationship should reflect the actual use-case semantics, not merely whether a feature is described as optional in ordinary language.

3. Why are failure states important?

Failure states clarify what the system does when an operation cannot complete successfully.

For example, a payment might enter a failed state rather than incorrectly progressing to a paid state.

Explicit failure behavior also helps developers identify which operations may be retried, which actions are prohibited, and which recovery procedures are required.

4. What should a technical design document say about uncertain requirements?

It should identify the uncertainty, explain its impact on the design, and record any assumptions being used.

If the uncertainty materially affects architecture, the document should identify the decision that needs to be resolved and who or what will resolve it.

5. How should two caching solutions be compared?

Define the actual requirements first.

Compare factors such as:

- Read and write performance under the expected workload.
- Data persistence and eviction behavior.

- Consistency requirements.
- Memory usage.
- Operational complexity.
- Failure recovery.
- Cost.
- Team familiarity.

Benchmarks should reflect realistic workloads rather than isolated, unrelated performance claims.

6. When should an ADR be reconsidered?

Reconsider it when its assumptions no longer hold, requirements change, measured behavior reveals a problem, operational constraints change, or a better-supported alternative becomes available.

An ADR should preserve the reasoning behind the original decision even if a later ADR supersedes it.

7. Why can a database migration be difficult to roll back?

The new application may already have written data using the new schema. The old application might not understand that data.

Restoring a backup could also erase legitimate changes made after the backup.

A migration plan must therefore consider schema compatibility, data transformation, write synchronization, backups, and recovery procedures.

8. How do UML diagrams and ADRs complement one another?

UML diagrams represent aspects of the system's structure and behavior.

ADRs record why important architectural choices were made.

For example, an activity diagram might show asynchronous notification processing, while an ADR explains why the team selected a queue-based approach rather than synchronous delivery.

The diagram describes the design; the ADR preserves the reasoning behind the decision.

Conclusion

This week focused on advanced UML diagrams, technical design documentation, and architectural decision-making. We learned how use-case, activity, and state machine diagrams represent system requirements, workflows, and state transitions. Technical design documents help define system scope, architecture, security, and reliability. Evaluating alternatives and planning rollouts and migrations help build reliable systems. Finally, the e-commerce case study showed how these concepts work together to create clear, maintainable, and well-documented software designs.

ONTO WEEK 5